

TÀI LIỆU ÔN TẬP

VẤN ĐÁP KỸ THUẬT

HiBuddy Application

Swipe & Discover

Matching Algorithm

Admin / Moderation

MỤC LỤC

PHẦN	TÊN PHẦN	NỘI DUNG
0	Kiến Trúc Tổng Quan	Stack, Folder Structure, Database
	Flow Swipe / Discover	Discover cards, Swipe action, DB Schema
2	Thuật Toán Tính Điểm	Project score, User score, Score blending
	Milvus Vector Database	Collections, Upsert vector, Hybrid search
4	Admin & Moderation	Report, Block, Approve/Reject project
	Xác Thực Email / OTP	Register, OTP verify, Rate limiting
6	Testing (pytest)	test_matching, test_swipe, test_email
	Q&A; Câu Hỏi Thắc	Q1-Q6 kịch bản trả lời đầy đủ
8	Bảng Số Liệu Quan Trọng	Constants, Weights cheatsheet

PHẦN 0 — KIẾN TRÚC TỔNG QUAN

0.1 Tech Stack

Layer	Công Nghệ	Ghi Chú
Backend Framework	FastAPI (Python)	async/await, Python 3.11+
SQL Database	PostgreSQL	async via asyncpg + SQLAlchemy
Vector Database	Milvus	Port 19530, pymilvus
Cache / Queue	Redis	Rate limiting, session cache
File Storage	MinIO	S3-compatible (ảnh, tài liệu)
AI Model	SentenceTransformer all-MiniLM-L6-v2	Local, offline, vector 384 chiều
Push Notify	FCM – Firebase Cloud Messaging	Mobile push notifications
Email	aiosmtplib	Async SMTP, gửi OTP

0.2 Cấu trúc thư mục Backend

Folder Structure

```
backend/  
├── app/  
│   ├── main.py           ← FastAPI app, gắn routers  
│   ├── config.py         ← Tất cả settings (env vars)  
│   ├── database.py        ← AsyncSession, engine  
│   ├── milvus_client.py   ← Kết nối & khởi tạo Milvus  
│   ├── api/  
│   │   ├── swipe.py      ← Router: /api/v1/swipe/*  
│   │   └── admin.py      ← Router: /api/v1/admin/*  
│   ├── services/  
│   │   ├── swipe_service.py ← Logic discover + match (1479 dòng!)  
│   │   ├── matching_service.py ← Thuật toán tính điểm Deterministic  
│   │   └── embedding_service.py ← AI vector (SentenceTransformer)  
│   └── models/  
│       ├── swipe.py       ← SwipeAction, Match, SwipeQueueItem  
│       └── trust_safety.py ← UserBlock, Report  
└── tests/                 ← pytest unit tests
```

0.3 Database Overview

□ 2 Database chạy song song: PostgreSQL (data nghiệp vụ) + Milvus (vector embeddings)

Database	Type	Nội dung chính
PostgreSQL	Relational SQL	20+ bảng: users, projects, swipe_actions, matches, reports, ...
Milvus	Vector DB	2 collections: user_profile_vectors, project_vectors (dim=384)

Các bảng SQL quan trọng:

- users, user_profiles, user_roles, user_skills, user_interests
- projects, project_role_slots, project_role_skill_requirements, project_members

- swipe_actions, matches, swipe_queue_items
- user_blocks, reports
- account_tokens, refresh_tokens, auth_identities
- notifications, chats, messages, outbox_events, admin_audit_logs

PHẦN 1 — FLOW SWIPE / DISCOVER

1.1 Load Danh Sách Swipe (GET /api/v1/swipe/discover)

□ Endpoint: GET /api/v1/swipe/discover?mode=CONTRIBUTOR&limit=20
File: app/api/swipe.py → discover_cards() → swipe_service.py → get_discover_cards()

1	API Layer	swipe.py::discover_cards() → Validate limit (1–50) → Gọi swipe_service
2	Phân loại Mode	CONTRIBUTOR → _discover_projects() OWNER → _discover_users()
3A	Build Exclusion List	_get_exclusion_ids() → Truy SQL lấy danh sách KHÔNG hiển thị
3B	AI Vector Search	_search_similar_projects() → Encode user vector → Query Milvus
3C	SQL Query	SELECT projects WHERE status=RECRUITING AND review_status=APPROVED AND ...
3D	Tính Điểm	matching_service::calculate_project_score_details() cho từng project
3E	Score Blending	final_score = (deterministic_score + vec_score×100) / 2
3F	Sort & Pagination	Sort giảm dần theo score, cursor-based pagination
4	Return JSON	{ project_cards, next_cursor, daily_likes_remaining, daily_superlikes_remaining }

Chi tiết Bước 3A: Exclusion List (_get_exclusion_ids)

Loại trừ	Điều kiện
PASS (7 ngày)	swipe_actions WHERE action='PASS' AND created_at > now()-7days
LIKE / SUPER_LIKE	swipe_actions WHERE action IN ('LIKE','SUPER_LIKE') (mọi thời gian)
Đang trong Queue	swipe_queue_items WHERE is_active=True
Đã Match	matches WHERE is_unmatched=False
Đã Block (2 chiều)	user_blocks WHERE blocker=user OR blocked=user

Chi tiết Bước 3B: Milvus Vector Search

Milvus Search Query

```
# 1. Build text từ user profile
text = "full_name | bio | roles | skills(level) | university"

# 2. Encode → vector 384 chiều
vector = SentenceTransformer("all-MiniLM-L6-v2").encode(text)

# 3. Search Milvus collection "project_vectors"
results = collection.search(
    data=[vector],
    anns_field="vector",
    param={"metric_type": "COSINE", "params": {"ef": 64}},
    limit=limit*2,          # lấy gấp đôi để dự phòng
    expr="status == 'RECRUITING'",
    output_fields=["project_id"]
)
```



Công thức Cosine Similarity (tự implement)

Cosine Similarity

```
dot_product = sum(a[i] * b[i] for i in range(384))
norm_a      = sqrt(sum(a[i]**2 for i in range(384)))
norm_b      = sqrt(sum(b[i]**2 for i in range(384)))
cosine      = dot / (norm_a * norm_b) # clamp → [0.0, 1.0]
```

PHẦN 1 (tiếp) — FLOW SWIPE ACTION

1.2 Thực Hiện Swipe (POST /api/v1/swipe/action)

- Endpoint: POST /api/v1/swipe/action
Body: { target_type: "PROJECT"|"USER", target_id, action: "PASS"|"LIKE"|"SUPER_LIKE" }
File: app/services/swipe_service.py → perform_swipe_action()

B1	Validate Input target_type ∈ {PROJECT, USER}, action ∈ {PASS, LIKE, SUPER_LIKE}
B2	Validate Target _validate_swipe_target() → kiểm tra tồn tại, status, không tự swipe mình, UserBlock
B3	Check Trùng Lặp SELECT swipe_actions WHERE swiper_id+target+context+is_active=True
B4	Giới Hạn Ngày LIKE: max 50/ngày SUPER_LIKE: max 3/ngày → raise 400 nếu vượt
B5	Lưu DB INSERT SwipeAction → db.flush()
B6	PASS Early Return Nếu action=PASS → return {matched: False} ngay
B7	Check Match _check_match() → 2 chiều: cả contributor và owner đều LIKE nhau chưa?
B8	Tạo Match _create_match() → INSERT: matches + chats + 2 notifications + 2 outbox_events
B9	Super Like Notify Nếu SUPER_LIKE nhưng chưa match → _notify_super_like() → FCM

Logic Check Match 2 Chiều

Match Logic

```
# Contributor LIKE project:
# → Kiểm tra owner đã LIKE contributor này chưa?
SELECT swipe_actions WHERE
    swiper_id = project.owner_id
    AND target_type = "USER"
    AND target_id = contributor.id
    AND action IN ("LIKE", "SUPER_LIKE")
    AND context_project_id = project.id
    AND is_active = True

# Nếu có → MATCH! → _create_match()
# INSERT: matches + chats + notifications (x2) + outbox_events (x2)
```

1.3 Schema Bảng Dữ Liệu Swipe

Bảng	Cột Quan Trọng	Ghi Chú
swipe_actions	swiper_id, target_type, target_id, action, context_project_id, context_key, is_active	UNIQUE on (swiper_id, target_type, target_id, context_key) WHERE is_active=True

matches	user_id (contributor), project_id, owner_id, match_score, role_matched, is_unmatched	UNIQUE on (user_id, project_id) WHERE is_unmatched=False
swipe_queue_items	expires_at (24h TTL), is_active, resolution	resolution: PASS LIKE SUPER_LIKE EXPIRED REMOVED

PHẦN 2 — THUẬT TOÁN TÍNH ĐIỂM (matching_service.py)

2.1 Tính Điểm Project (Contributor xem Project)

Hàm: calculate_project_score_details(user, project, owner)

□ slot_fit chiếm 80% tổng điểm → đây là yếu tố quan trọng nhất!

Yếu Tố	Trọng Số	Cách Tính
slot_fit (role + skill)	× 0.80	best_role_slot() → role_score×0.80 + skill_score×0.20
interest_score	× 0.08	Jaccard Index(user.interests, project.fields) × 100
owner_quality	× 0.08	reputation_pct + 15 bonus nếu verified_student (max 100)
recency	× 0.04	<7 ngày=100, <14=70, <30=40, ≥30=10

Công thức tổng hợp điểm Project

score = slot_fit * 0.80
+ interest_score * 0.08
+ owner_quality * 0.08
+ recency * 0.04

Chi tiết tính slot_fit

Sub-yếu tố	Trọng số trong slot	Logic
role_score	× 0.80	normalize_name(user.roles) == normalize_name(slot.role_name) → 100 else 0 VD: "Android Developer" → alias → "mobile developer"
skill_score	× 0.20	(chỉ tính khi role match) fuzzy_match(required_skill, user_skills, threshold=85%) required=True → weight=2.0 optional → weight=1.0 skill_score = earned/total × 100

Ví dụ minh họa:

Ví dụ tính slot_fit

User: "Backend Developer" | skills: Python (ADVANCED), PostgreSQL (INTERMEDIATE)
Project slot: "Backend Developer" needs:
→ Python (INTERMEDIATE, required=True, weight=2.0) ← matched □
→ PostgreSQL (BEGINNER, optional=False, weight=1.0) ← matched □

role_score = 100 (match)
skill_score = (2.0+1.0) / (2.0+1.0) × 100 = 100.0
slot_fit = 100×0.80 + 100×0.20 = 100.0

2.2 Tính Điểm User (Owner xem Ứng Viên)

Hàm: calculate_user_score_details(owner, target_user, project)

Yếu Tố	Trọng Số	Cách Tính
project_score	× 0.70	Gọi lại calculate_project_score_details(target_user, project, owner)
reputation_pct	× 0.12	target_user.reputation_score / 5.0 × 100
experience_pct	× 0.08	min(projects_completed / 10, 1.0) × 100

verified_student	× 0.10	100.0 nếu verified_student=True, else 0.0
------------------	--------	---

2.3

Score Blending (AI + Deterministic)

Score Blending

```
if ENABLE_EMBEDDINGS:
    vec_score = cosine_similarity(user_vec, project_vec) × 100
    final_score = (deterministic_score + vec_score) / 2
else:
    final_score = deterministic_score
```

Ngưỡng	Nhãn
≥ 85	Excellent 🌟
≥ 70	Very Good 🌟
≥ 50	Good 🌟
≥ 30	Fair
< 30	Low

📌 Newbie Boost: User < 30 ngày HOẶC chưa làm project nào

→ reputation giả định 3.8/5.0 = 76% (thay vì 0)

→ Tránh new user bị điểm thấp ngay từ đầu

PHẦN 3 — MILVUS VECTOR DATABASE

3.1 Cấu Trúc Collections

Field	Type	Chi tiết
Collection: user_profile_vectors		
id	INT64	Primary key, auto_id=True
user_id	VARCHAR(36)	UUID dạng string
vector	FLOAT_VECTOR(384)	Output của all-MiniLM-L6-v2
mode	VARCHAR(20)	"CONTRIBUTOR" "OWNER" "BOTH"
is_active	BOOL	—
reputation_score	FLOAT	—
Collection: project_vectors		
project_id	VARCHAR(36)	UUID
vector	FLOAT_VECTOR(384)	—
field	VARCHAR(50)	AI, Web, Mobile, ...
status	VARCHAR(20)	"RECRUITING" "CLOSED"
work_mode	VARCHAR(20)	REMOTE / ONSITE / ...

- Index: HNSW trên field "vector" | M=16, efConstruction=200 | Metric: COSINE
Search param: ef=64 (accuracy vs speed trade-off)

3.2 Upsert Vector Khi Có Dữ Liệu Mới

upsert_project_vector()

```
# Chạy dưới dạng BackgroundTask (không block response)
async def upsert_project_vector(project):
    # 1. Build text từ project
    text = f"{title} | {description} | {field} | {role_names} | {work_mode}"

    # 2. Encode → vector 384 chiều
    vector = SentenceTransformer("all-MiniLM-L6-v2").encode(text).tolist()

    # 3. Xóa embedding cũ (nếu có)
    if project.embedding_id:
        collection.delete(f"id == {project.embedding_id}")

    # 4. Insert embedding mới
    result = collection.insert([{"project_id": str(project.id), "vector": vector, ...}])
    collection.flush() # ghi xuống disk

    # 5. Lưu embedding_id mới vào PostgreSQL
    project.embedding_id = result.primary_keys[0]
```

3.3 Luồng Hybrid Search (Milvus + PostgreSQL)

- Tại sao phức tạp? → Milvus KHÔNG biết lịch sử swipe, PostgreSQL KHÔNG làm được semantic search
→ Phải kết hợp 2 nguồn thủ công (choreograph)

[1] Encode: SentenceTransformer → user_vector (384 chiều)

[2] Milvus Query: top (limit×2=40) project có COSINE similarity cao nhất, filter status=RECRUITING

[3] Filter: Loại bỏ IDs có trong exclude_project_ids (từ swipe history PostgreSQL)

[4] SQL Query: SELECT projects WHERE id IN (filtered_ids) AND review_status=APPROVED AND ...

[5] Deterministic Score: matching_service → tính điểm cho từng project

[6] Vec Score: $\cosine(\text{user_vec}, \text{project_vec}) \times 100$ (re-compute, dùng _embedding_cache)

[7] Blend: $\text{final_score} = (\text{deterministic} + \text{vec_score}) / 2$

[8] Sort: Sắp xếp giảm dần theo final_score

- Tại sao tính cosine lại lần 2?
→ Milvus trả về distance nhưng đã mất sau query
→ Cần tính lại để blend với deterministic score
→ Dùng in-memory cache _embedding_cache để tránh encode lại

PHẦN 4 — ADMIN & MODERATION

4.1 Cấu Trúc Dữ Liệu

Model	Cột Quan Trọng	Ghi Chú
Report	reporter_id, reported_id, reason (500), description (1000), status (PENDING/RESOLVED), evidence_url	File: models/trust_safety.py
UserBlock	blocker_id, blocked_id	UNIQUE(blocker_id, blocked_id) CHECK(blocker_id ≠ blocked_id)
Project	review_status (APPROVED/FLAGGED/REJECTED), moderation_categories, moderation_reasons	Default: "APPROVED" → xuất hiện ngay

4.2 Flow Admin Duyệt Project

- Project mới tạo: review_status = "APPROVED" (default) → xuất hiện ngay trong Discover
→ Khi bị report đủ lần → chuyển "FLAGGED" → Admin mới cần can thiệp

Endpoint	Action	Flow
GET /admin/projects/flagged	List cần duyệt	SELECT projects WHERE review_status=FLAGGED ORDER BY created_at DESC (yêu cầu Admin role)
POST /admin/projects/{id}/approve	APPROVE	review_status=APPROVED → INSERT Notification → INSERT AdminAuditLog → INSERT OutboxEvent (FCM)
POST /admin/projects/{id}/reject	REJECT	review_status=REJECTED + status=CLOSED → Notification + AuditLog + OutboxEvent (FCM)

4.3 UserBlock – Tác Động Lên Hệ Thống

Nơi áp dụng	Logic
Khi tải Discover (<code>_get_exclusion_ids</code>)	SELECT user_blocks WHERE blocker_id=user OR blocked_id=user → Thêm vào exclude_user_ids (2 chiều) → Block người nào → họ biến mất khỏi Explore của nhau
Khi thực hiện Swipe (<code>_validate_swipe_target</code>)	SELECT user_blocks WHERE (blocker=me AND blocked=target) OR (blocker=target AND blocked=me) → Nếu có block record → raise ValueError("Target is unavailable") → Không thể swipe người đã block

4.4 Discovery Filter

SQL Filter trong `_discover_projects()`

```
# swipe_service.py → _discover_projects() line ~938
.where(
    Project.status      == "RECRUITING",
    Project.review_status == "APPROVED",    # ← CHỈ hiện APPROVED
    Project.owner_id    != user.id,
    User.is_active      == True,
    User.email_verified == True,
    Project.id.not_in(exclude_project_ids)
)
```

PHẦN 5 — XÁC THỰC EMAIL / OTP SINH VIÊN

5.1 Flow Đăng Ký + Gửi OTP

□ Endpoint: POST /api/v1/auth/register | File: app/services/auth_service.py

- [1] **Validate**: Email/username không trùng trong DB
- [2] **Hash Password**: `bcrypt.hash(password)` → lưu vào DB
- [3] **INSERT User**: users table, email_verified=False
- [4] **INSERT Profile**: user_profiles: display_name, reputation_score=0.0, projects_completed=0
- [5] **Tạo OTP**: `_create_account_code()` → `generate_numeric_code()` (6 chữ số) → `bcrypt hash` → INSERT account_tokens (TTL 15 phút)
- [6] **Gửi Email**: BackgroundTask: `aiosmtplib.send()` → không block response
- [7] **Return JWT**: access_token + refresh_token + requires_email_verification=True

5.2 Flow Verify OTP

□ Endpoint: POST /api/v1/auth/verify-email | OTP chỉ dùng được 1 lần (1-time use)

Logic _consume_account_code()

```
# _consume_account_code(db, email, purpose, code):
1. SELECT account_tokens WHERE email=email AND purpose=EMAIL_VERIFICATION AND consumed_at IS NULL
2. Kiểm tra: token.expires_at <= now → raise 400 "Invalid or expired code"
3. Kiểm tra: token.attempts >= 5 → raise 429 (too many attempts)
4. So sánh: hash_token(code) == token.code_hash
   → Sai: token.attempts += 1, commit, raise 400
   → Đúng: token.consumed_at = now (vô hiệu hóa)
       user.email_verified = True
```

5.3 Rate Limiting OTP

Giới Hạn	Giá Trị	Behavior
OTP TTL	15 phút	AUTH_CODE_TTL_MINUTES = 15
Max sai	5 lần	AUTH_CODE_MAX_ATTEMPTS = 5 → sau đó OTP bị consumed ngay
Cooldown gửi lại	60 giây	AUTH_CODE_RESEND_SECONDS = 60 → raise HTTP 429 + header "Retry-After: N"

5.4 Feature Gating

Flag	Ý Nghĩa	Tác Động
email_verified = True	Email xác thực qua OTP	Bắt buộc mới được xuất hiện trong Discover của Owner
verified_student = True	Admin duyệt thẻ sinh viên	+10 điểm trong calculate_user_score, +15 điểm bonus owner_quality

- email_verified ≠ verified_student:
- email_verified: xác nhận email thật (OTP tự động)
 - verified_student: xác nhận sinh viên thật (Admin duyệt thẻ SV)

PHẦN 6 — TESTING (pytest)

□ Chạy: `cd backend && pytest tests/ -v` | File: `backend/tests/`

□ `test_matching.py`

- `test_role_aliases_are_normalized`: "Android Developer" → "mobile developer", "Full-Stack" → "fullstack developer"
- `test_matching_uses_skills_from_the_matched_role_only`: không lấy skill của role khác (VD: Figma không áp vào Backend slot)
- `test_newbie_bootstrap_reputation`: user < 30 ngày → reputation giả định $3.8/5.0 = 76\%$
- `test_verified_student_influence`: verified owner → +15 bonus | verified user → `factors["verified_student"] = 100.0`

□ `test_swipe_project.py`

- `test_swipe_project_scenarios_with_varied_owner_quality`: 5 kịch bản
- Assert thứ tự: `high_fit_high_owner > high_fit_low_owner`
- Assert: `no_fit` → điểm thấp dù owner tốt đến đâu

□ `test_email_verification.py`

- `test_email_verification_flow`: Tạo OTP 6 chữ số → verify sai email → HTTP 403 → verify đúng → `email_verified=True`, OTP consumed
- `test_register_duplicate_email_or_username_returns_conflict`: HTTP 409
- `test_public_resend_verification`: AccountToken insert + email gửi đúng + code 6 chữ số

□ `test_swipe_queue.py` & `test_swipe_user.py`

- Queue 24h TTL: card vào queue → sau 24h tự PASS
- Giới hạn queue: tối đa 3 card/loại
- Owner swipe user trong context project cụ thể

PHẦN 7 — CÂU HỎI THẦY & KỊCH BẢN TRẢ LỜI

❑ Q1: Phần khó nhất về kỹ thuật là gì?

Dạ, phần khó nhất là hàm `get_discover_cards` trong `swipe_service.py`, cụ thể là bài toán Hybrid Query.

- (1) Tính vector 384 chiều của user bằng `SentenceTransformer`.
- (2) Query Milvus → top 40 project ngữ nghĩa gần nhất (COSINE + HNSW index).
- (3) Cùng lúc, query PostgreSQL để lấy exclusion list (đã swipe, block, queue).
- (4) Lọc kết quả Milvus theo exclusion list PostgreSQL.
- (5) Tính deterministic score + blend với vector cosine score.

Cái khó: 2 database không nói chuyện trực tiếp được. Phải choreograph thủ công.

Fallback: Nếu Milvus down → graceful fallback về PostgreSQL-only mode.

❑ Q2: Nếu thêm dữ liệu mới thì xử lý sao?

Khi user tạo project mới hoặc cập nhật profile, hệ thống xử lý 2 bước:

BƯỚC 1 - Sync: Lưu text data vào PostgreSQL ngay lập tức. `embedding_id = None`.

BƯỚC 2 - Async (`BackgroundTask`): `upsert_project_vector()`:

→ `build_project_text()` → `encode_text()` → `SentenceTransformer` → `vector[384]`

→ Milvus: `delete old` → `insert new` → `flush()`

→ PostgreSQL: `update projects.embedding_id = new_milvus_id`

Tại sao async? Encode AI model tốn ~100-200ms, không muốn user đợi.

Trong thời gian chờ: project vẫn xuất hiện (SQL filter), nhưng chưa được AI xếp hạng chuẩn.

❑ Q3: Database có bao nhiêu bảng, bao nhiêu DB?

2 database chạy song song:

PostgreSQL (data nghiệp vụ): 20+ bảng.

Milvus (vector): 2 collections – `user_profile_vectors`, `project_vectors`.

Mỗi collection: HNSW index, COSINE metric, vector 384 chiều.

❑ Q4: Chỉ ra flow cho phần Swipe - từng file?

[1] `app/api/swipe.py` → `discover_cards()`: nhận request, validate

[2] `app/services/swipe_service.py` → `get_discover_cards()`: phân loại mode

[3] `app/services/swipe_service.py` → `_get_exclusion_ids()`: SQL exclusion list

[4] `app/services/swipe_service.py` → `_search_similar_projects()`: Milvus query

[5] `app/services/swipe_service.py` → `_discover_projects()`: SQL + score

[6] `app/services/matching_service.py` → `calculate_project_score_details()`: tính điểm

[7] `app/services/swipe_service.py` (line ~966): Score Blending → `final_score`

❑ Q5: Cái nút này gọi hàm nào?

Nút "Explore" / Load cards: GET /api/v1/swipe/discover → discover_cards() → get_discover_cards()

Nút "Like" (tim): POST /api/v1/swipe/action {action:"LIKE"} → swipe_action() → perform_swipe_action()

Nút "Pass" (x): POST /api/v1/swipe/action {action:"PASS"} → return ngay sau lưu PASS

Nút "Super Like" (sao): POST /api/v1/swipe/action {action:"SUPER_LIKE"} → lưu + _notify_super_like()

❑ Q6: Có test các chức năng này không?

Dạ có, dùng pytest. File trong backend/tests/.

test_matching.py: role alias, skill matching, newbie boost, verified student.

test_swipe_project.py: 5 kịch bản score, so sánh thứ tự đúng.

test_email_verification.py: OTP flow, duplicate email 409, resend OTP.

test_swipe_queue.py: queue 24h TTL, giới hạn 3 card/loại.

PHẦN 8 — BẢNG SỐ LIỆ QUAN TRỌNG (Cheatsheet)

8.1 Giới Hạn Hệ Thống

Hằng Số (config.py)	Giá Trị	Ý Nghĩa
SWIPE_DAILY_LIKE_LIMIT	50	Số LIKE tối đa mỗi ngày
SWIPE_DAILY_SUPERLIKE_LIMIT	3	Số SUPER_LIKE tối đa mỗi ngày
PASS_COOLDOWN_DAYS	7 ngày	Sau khi PASS, cooldown 7 ngày mới thấy lại
QUEUE_TTL	24h	SwipeQueueItem tự PASS sau 24h
QUEUE_LIMIT_PER_TYPE	3	Tối đa 3 card mỗi loại trong queue
AUTH_CODE_TTL_MINUTES	15 phút	OTP hết hạn sau 15 phút
AUTH_CODE_MAX_ATTEMPTS	5 lần	Nhập sai tối đa 5 lần, sau đó OTP bị consumed
AUTH_CODE_RESEND_SECONDS	60 giây	Cooldown giữa các lần gửi lại OTP

8.2 Thông Số Milvus

Thông Số	Giá Trị
EMBEDDING_DIM	384 (output của all-MiniLM-L6-v2)
Milvus Port	19530
Index Type	HNSW
HNSW M	16
efConstruction	200
ef (search)	64
Metric	COSINE
Collections	user_profile_vectors, project_vectors

8.3 Trọng Số Điểm

Điểm Project (Contributor → Project)	Điểm User (Owner → Ứng Viên)
slot_fit (role + skill) 80%	project_score 70%
interest_score 8%	reputation_pct 12%
owner_quality 8%	experience_pct 8%
recency 4%	verified_student 10%

8.4 Rank Labels

Ngưỡng	Nhãn	Màu
≥ 85	Excellent ■	Xanh lá ■■■m

≥ 70	Very Good ■	Xanh d■■■ng
≥ 50	Good ■	Xanh lá nh■■t
≥ 30	Fair	Cam
< 30	Low	■■

- `ENABLE_MILVUS = True (config.py) | ENABLE_EMBEDDINGS = True (config.py)`
Project mới: `review_status = "APPROVED"` (default) → xuất hiện ngay
Phân quyền Admin: `get_current_admin()` middleware trong `admin.py`